UNIJUÍ — Universidade Regional do Noroeste do Estado do Rio Grande do Sul
Linguagens Formais e Compiladores

ESTUDO INDEPENDENTE 03

Analisador léxico da Padolang

Especificação com expressões regulares, resolução de conflitos, implementação e validação

Estudantes	Theodoro Gaspar Ferreira Vinícius Andrey Costa Mattos
Componente curricular	Linguagens Formais e Compiladores
Professor	Marcos Ronaldo Melo Cavalheiro
Atividade	Estudo Independente 03

Outubro de 2026

Sumário

1 Introdução

2 Problema 1 — Especificação formal dos tokens
alfabeto e notação; palavras reservadas; identificadores; números; operadores; atribuição e delimitadores; brancos e comentários; tabela de tokens

3 Problema 2 — Resolução de conflitos e tratamento léxico
prioridade de reconhecimento; maior casamento; controle de posição

4 Problema 3 — Implementação do analisador léxico
organização; execução; leitura; varredura; tratamento de erros

5 Problema 4 — Casos de teste e evidências de validação
teste válido; teste com erros léxicos; testes automatizados; conferência da especificação com o JFlex

6 Considerações finais

Referências

Declaração de uso de IA generativa

Apêndice A — Código-fonte do analisador (`Lexer.java`)

1 Introdução

A Padolang é a linguagem de programação criada pelo grupo na disciplina. Seus programas têm a extensão `.padoin`, as palavras reservadas são formadas a partir de *pad*, *pado* e *padoin*, e o emoji 👍 termina cada comando, no lugar do ponto e vírgula. O compilador é escrito em Java 21, sem bibliotecas externas, e traduz a Padolang para C:

```
arquivo.padoin → LÉXICO → tokens → SINTÁTICO → AST → SEMÂNTICO → código C
```

Este relatório trata da primeira etapa. O analisador léxico lê os caracteres do programa-fonte, agrupa-os em *lexemas* e entrega ao analisador sintático uma sequência de *tokens*; também descarta espaços em branco e comentários e associa as mensagens de erro à posição no fonte (AHO et al., 2008, seção 3.1). Três termos são usados em todo o texto, com o sentido de Aho et al. (2008, seção 3.1.2):

- **token**: o par formado pela classe (por exemplo, `NUM_INT`) e por um atributo, que aqui é o próprio lexema e a posição dele;
- **padrão**: a descrição da forma que os lexemas de um token podem ter, dada por uma expressão regular;
- **lexema**: a sequência de caracteres do fonte que casa com o padrão (por exemplo, `42`).

O analisador da Padolang é **escrito à mão** (a opção “manual” do enunciado): é o arquivo `src/padolang/lexer/Lexer.java`, um autômato finito determinístico codificado diretamente em Java. A especificação formal dos tokens foi escrita na notação do JFlex e, para garantir que as expressões regulares do relatório descrevem de fato o que o código reconhece, ela foi também executada: o scanner gerado pelo JFlex a partir dela lista exatamente os mesmos tokens que o analisador escrito à mão (Seção 5.4).

O analisador já existia no compilador. Para esta atividade ele recebeu o comentário de múltiplas linhas (`#* ... *#`), que a linguagem ainda não tinha, um erro próprio para números com mais de um ponto decimal e a indicação do lexema em cada erro da listagem de tokens.

Onde está cada item pedido no enunciado

Item	Seção do relatório	Arquivos no pacote entregue
Problema 1: expressões regulares	2	`docs/estudo-independente-03/jflex/padolang.flex`
Problema 2: conflitos e posição	3	`src/padolang/lexer/Lexer.java`
Problema 3: implementação	4 e Apêndice A	`src/padolang/lexer/`, `src/padolang/cli/Cli.java`
Problema 4: testes e evidências	5	`testes/lexer_editor/*.padoin` (entradas) e `*.tokens` (saídas)

2 Problema 1 — Especificação formal dos tokens

2.1 Alfabeto e notação

O fonte é lido em UTF-8 e tratado como uma sequência de *code points* Unicode. Fora de textos e comentários, o alfabeto de entrada Σ da Padolang é:

Grupo	Símbolos
Letras	`a` – `z`, `A` – `Z` e `_` (só ASCII: `ç`, `ã` etc. não pertencem a Σ)
Dígitos	`0` – `9`
Símbolos	`+ - * / = ! < > ( ) { } , . " \ #`
Emoji	👍 (U+1F44D), os modificadores de tom de pele (U+1F3FB a U+1F3FF) e o seletor de variação (U+FE0F)
Brancos	espaço, tabulação (`\t`), retorno de carro (`\r`) e quebra de linha (`\n`)

Dentro de um texto entre aspas ou de um comentário pode aparecer qualquer caractere Unicode. Qualquer outro símbolo fora desses dois contextos é um erro léxico.

As expressões regulares estão na **notação do JFlex**. Os elementos usados são: `"abc"`, a cadeia literal; `[a-z]`, uma classe de caracteres; `[^a]`, qualquer caractere fora da classe; `r*`, `r+` e `r?`, zero ou mais, uma ou mais e zero ou uma ocorrência; `r | s`, a alternativa; `\x`, o caractere `x` sem significado especial; `\u{1F44D}`, o *code point* indicado; e `{NOME}`, o uso de uma definição regular. Duas definições servem de base para as demais (AHO et al., 2008, seção 3.3.4):

```
LETRA  = [a-zA-Z_]
DIGITO = [0-9]
```

2.2 a) Palavras reservadas

A Padolang tem 23 palavras reservadas. Cada uma é uma cadeia fixa, então o padrão de cada token é a própria cadeia, e o conjunto todo é a união finita delas:

```
RESERVADA = "PADOIN" | "padoin" | "padoINTER" | "padoFLOAT" | "padoOLEAN" | "padoSTRING"
          | "padoIN" | "padoPRINT" | "padoPRINTLN" | "pad" | "oin" | "papapadoin"
          | "padoinquanto" | "padBREAK" | "padCONTINUE" | "padUNCTION" | "padOUT"
          | "padoTRUE" | "padoFALSE" | "padAND" | "padOR" | "padNOT" | "padoMOD"
```

Lexema	Token	Significado	Lexema	Token	Significado
`PADOIN`	`PROGRAMA`	bloco principal	`padoinquanto`	`ENQUANTO`	laço “enquanto”
`padoin`	`DECLARA`	declaração de variável	`padBREAK`	`INTERROMPE`	sai do laço
`padoINTER`	`TIPO_INTER`	tipo inteiro	`padCONTINUE`	`PROSSEGUE`	próxima volta do laço
`padoFLOAT`	`TIPO_FLOAT`	tipo real	`padUNCTION`	`FUNCAO`	definição de função
`padoOLEAN`	`TIPO_OLEAN`	tipo lógico	`padOUT`	`RETORNO`	retorno de função
`padoSTRING`	`TIPO_STRING`	tipo texto	`padoTRUE`	`VERDADEIRO`	constante lógica
`padoIN`	`LEITURA`	entrada	`padoFALSE`	`FALSO`	constante lógica
`padoPRINT`	`ESCRITA`	saída	`padAND`	`E`	“e” lógico
`padoPRINTLN`	`ESCRITA_LINHA`	saída com quebra de linha	`padOR`	`OU`	“ou” lógico
`pad`	`SE`	condicional	`padNOT`	`NAO`	negação lógica
`oin`	`SENAO`	senão	`padoMOD`	`MODULO`	resto da divisão
`papapadoin`	`REPETICAO`	laço “para”			

Justificativa. Cada palavra tem um token próprio, porque a gramática trata cada uma de modo diferente. Na classificação de Price e Toscani (2001, seção 2.2), as palavras reservadas, os operadores e os delimitadores são *tokens simples*: a classe descreve o token por completo. Identificadores, números e textos são *tokens com argumento*, e o argumento é o lexema. As palavras diferenciam maiúsculas de minúsculas: `PADOIN`, `padoin` e `padoIN` são três tokens. Todas elas também casam com o padrão de identificador da Seção 2.3, e várias são prefixo de outras (`pad`, `padoin`, `padoinquanto`); como esses dois conflitos são resolvidos está na Seção 3.

2.3 b) Identificadores

```
ID = {LETRA} ({LETRA} | {DIGITO})*
```

Um identificador nomeia uma variável, um parâmetro ou uma função. As regras são:

- começa por uma letra ASCII ou por `_`, e continua com letras, dígitos e `_`, sem limite de tamanho;
- **não começa por dígito.** Assim o primeiro símbolo já separa as duas classes: letra inicia identificador ou palavra reservada, dígito inicia número. O autômato decide o caminho sem precisar voltar atrás;
- **só letras ASCII.** `ação` não é um identificador. A restrição mantém Σ pequeno e evita nomes visualmente iguais com códigos diferentes; como o compilador gera C, também garante que todo identificador da Padolang é um identificador válido no C gerado;
- diferencia maiúsculas de minúsculas (`total` e `Total` são nomes distintos);
- não pode ser igual a uma palavra reservada. Ter uma palavra reservada como prefixo é permitido: `padoinho` e `pad_2` são identificadores.

Válidos	Inválidos
`x`, `nome_1`, `_escapes`, `ehPar`, `padoinho`	`2x` (começa por dígito), `ação` (letras fora de Σ), `pad` (palavra reservada), `a-b` (são três tokens)

2.4 c) Números inteiros e reais

```
NUM_INT   = {DIGITO}+
NUM_FLOAT = {DIGITO}+ "." {DIGITO}+
```

Justificativa.

- Um inteiro é uma sequência de um ou mais dígitos. Zeros à esquerda são aceitos (`007` é o inteiro 7).
- Um real tem **exatamente um ponto**, com pelo menos um dígito de cada lado. A expressão só tem uma ocorrência de `"."`, e é isso que impõe o limite de um ponto decimal: `1.2.3` não casa. Exigir dígitos dos dois lados exclui `5.` e `.5`, que são fáceis de confundir com um inteiro seguido de pontuação.
- O sinal não faz parte do número. Em `-3`, o `-` é o operador `MENOS`, e é o analisador sintático que o trata como menos unário. Se o sinal entrasse no padrão, `a-1` seria lido como `a` seguido de `-1`, sem operador.
- Não há notação com expoente (`1e5`) nem sufixos.
- Se o valor cabe no tipo (32 bits para `padoINTER`) é verificado na análise semântica; o léxico só reconhece a forma.

Na forma de autômato finito determinístico $M = (\Sigma, Q, \delta, q_0, F)$ (MENEZES, 2011), os dois padrões juntos têm $Q = \{q_0, q_1, q_2, q_3\}$ e $F = \{q_1, q_3\}$, com as transições da Figura 1. O estado q_2, depois do ponto, não é final: o lexema `3.` não é aceito. As setas tracejadas não fazem parte do autômato dos padrões; elas mostram onde o analisador acusa erro em vez de simplesmente parar (Seção 4.5).

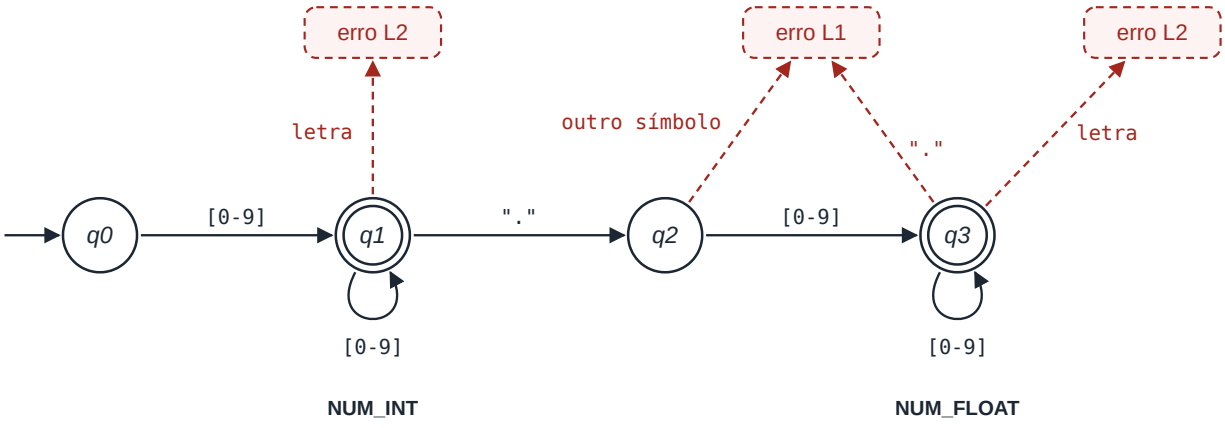


Figura 1 — Diagrama de transições dos números. Estados com círculo duplo são finais.

2.5 d) Operadores aritméticos, relacionais e lógicos

Todo operador é uma cadeia fixa, de um ou dois símbolos ou em forma de palavra reservada.

Grupo	Expressão regular	Tokens
Aritméticos	`"+" | "-" | "*" | "/"`	`MAIS`, `MENOS`, `VEZES`, `DIVIDIDO`
Incremento e decremento	`"++" | "--"`	`INCREMENTO`, `DECREMENTO`
Resto da divisão	`"padoMOD"`	`MODULO`
Relacionais	`"==" | "!=" | "<" | "<=" | ">" | ">="`	`IGUAL`, `DIFERENTE`, `MENOR`, `MENOR_IGUAL`, `MAIOR`, `MAIOR_IGUAL`
Lógicos	`"padAND" | "padOR" | "padNOT"`	`E`, `OU`, `NAO`

Justificativa. Os operadores lógicos e o resto da divisão são palavras, por coerência com o tema da linguagem; por isso `%`, `&` e `|` não pertencem a Σ. O `!` só existe como primeiro símbolo de `!=`: sozinho é erro léxico, e a negação se escreve `padNOT`. Vários operadores são prefixo de outros (`+` e `++`, `=` e `==`, `<` e `<=`); a regra que escolhe entre eles está na Seção 3.2.

2.6 e) Atribuição, delimitadores e pontuação

Token	Expressão regular	Uso
`ATRIB`	`"="`	atribuição e valor inicial de uma declaração
`ABRE_PAR`, `FECHA_PAR`	`"("`, `")"`	agrupam expressões, condições, parâmetros e argumentos
`ABRE_CHAVE`, `FECHA_CHAVE`	`"{"`, `"}"`	delimitam blocos
`VIRGULA`	`","`	separa parâmetros, argumentos e as partes do `papapadoin`
`JOINHA`	`\u{1F44D} [\u{1F3FB}-\u{1F3FF}]? \u{FE0F}?`	termina um comando (o papel do `;` em C e Java)

Justificativa. A atribuição é `=`, como em Java e C; a Padolang não tem `:=`, e por isso `:` não pertence a Σ. O terminador de comando é o 👍. O padrão aceita um modificador de tom de pele e o seletor de variação depois dele, porque teclados e seletores de emoji costumam inserir esses *code points* sem que o programador perceba; 👍, 👍🏽 e 👍 são o mesmo token, sempre com o lexema 👍. O ponto e vírgula não existe na linguagem.

Os textos entre aspas completam os literais da linguagem. O conteúdo é qualquer caractere que não seja aspa, barra invertida ou quebra de linha, ou então uma das quatro sequências de escape:

```
ESCAPE = \\ [nt\"\\]
STRING = \" ( [^\"\\\n] | {ESCAPE} )* \"
```

2.7 f) Espaços em branco, quebras de linha e comentários

```
BRANCO    = [ \t\r\n]+
COM_LINHA = "#" ( [^*\n] [^\n]* )?
COM_BLOCO = "#*" ( [^*] | \*+ [^*#] )* \*+ "#"
```

Brancos. Espaço, tabulação, `\r` e `\n` não geram token: são consumidos e descartados. Eles ainda têm dois efeitos: separam lexemas (`padoin x` são dois tokens, `padoinx` é um) e a quebra de linha atualiza a contagem de linhas (Seção 3.3). A Padolang não usa indentação nem fim de linha como sintaxe.

Comentário de linha. Começa em `#` e vai até o fim da linha, sem incluir o `\n`.

Comentário de bloco. Vai de `#*` até o primeiro `*#` e pode ocupar várias linhas ou ficar no meio de uma. A expressão é construída para que o corpo nunca contenha `*#`: o corpo é uma repetição de (i) um símbolo que não é `*`, ou (ii) uma sequência de `*` seguida de um símbolo que não é `*` nem `#`; o fechamento é uma sequência de `*` seguida de `#`. Por essa expressão, `#**#` é um comentário completo e `#*#` não é, porque o único `*` pertence à abertura. A Figura 2 mostra o autômato dos dois comentários.

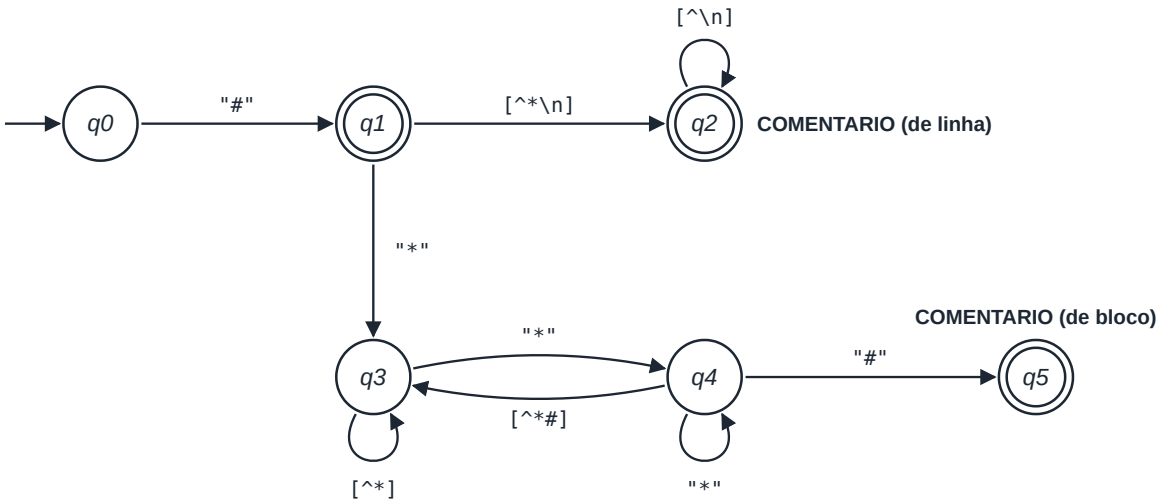


Figura 2 — Diagrama de transições dos comentários de linha e de bloco.

Justificativa.

- **O comentário de linha não pode começar por `#*`.** Por isso o segundo símbolo do padrão é `[^*\n]`, e não `[^\n]`. Com a forma mais simples `"#" [^\n]*`, os dois padrões casariam com `#* a *# x = 1`, e a regra do maior casamento escolheria o de linha, que consumiria também o `x = 1`. Com a exclusão, os dois padrões nunca casam com o mesmo prefixo: o símbolo depois do `#` decide.
- **O comentário de bloco não aninha.** Ele fecha no primeiro `*#`, como o `/* */` de C e Java. Para aceitar comentários aninhados seria preciso contar aberturas e fechamentos, o que um autômato finito não faz: a linguagem deixaria de ser regular (MENEZES, 2011) e não poderia ser descrita por uma expressão regular.
- Dentro de um texto, `#` e `#*` são conteúdo: `"#* a *#"` é um token `STRING`. Dentro de um comentário de linha, `#*` não abre um bloco.
- Os comentários não chegam ao analisador sintático. O analisador léxico pode, opcionalmente, entregá-los como tokens `COMENTARIO` (modo usado pelo formatador de código e pela listagem deste relatório).

2.8 Tabela de tokens

Todas as classes de token da Padolang (48, mais a classe `ERRO` dos trechos inválidos)

Categoria	Classes	Padrão
Palavras reservadas (19)	`PROGRAMA`, `DECLARA`, `TIPO_INTER`, `TIPO_FLOAT`, `TIPO_OLEAN`, `TIPO_STRING`, `LEITURA`, `ESCRITA`, `ESCRITA_LINHA`, `SE`, `SENAO`, `REPETICAO`, `ENQUANTO`, `INTERROMPE`, `PROSSEGUE`, `FUNCAO`, `RETORNO`, `VERDADEIRO`, `FALSO`	cadeia fixa (Seção 2.2)
Operadores em palavra (4)	`E`, `OU`, `NAO`, `MODULO`	cadeia fixa
Identificador	`ID`	`{LETRA} ({LETRA} | {DIGITO})*`
Literais	`NUM_INT`, `NUM_FLOAT`, `STRING`	Seções 2.4 e 2.6
Operadores aritméticos (6)	`MAIS`, `MENOS`, `VEZES`, `DIVIDIDO`, `INCREMENTO`, `DECREMENTO`	cadeia fixa
Atribuição e relacionais (7)	`ATRIB`, `IGUAL`, `DIFERENTE`, `MENOR`, `MENOR_IGUAL`, `MAIOR`, `MAIOR_IGUAL`	cadeia fixa
Delimitadores (6)	`ABRE_PAR`, `FECHA_PAR`, `ABRE_CHAVE`, `FECHA_CHAVE`, `VIRGULA`, `JOINHA`	cadeia fixa; 👍 com modificadores
Comentário	`COMENTARIO`	`{COM_LINHA} | {COM_BLOCO}`
Fim de arquivo	`EOF`	fim da entrada

A especificação completa, com as regras na ordem de prioridade e os padrões dos lexemas malformados, está no arquivo `padolang.flex` do pacote entregue.

3 Problema 2 — Resolução de conflitos e tratamento léxico

Com vários padrões, um mesmo trecho da entrada pode casar com mais de um. Geradores como o Lex e o JFlex resolvem isso com duas regras, aplicadas nesta ordem (AHO et al., 2008, seção 3.5.3):

1. **maior casamento:** entre os prefixos da entrada que casam com algum padrão, escolhe-se o mais longo;
2. **prioridade:** se o prefixo mais longo casa com dois ou mais padrões, vale o padrão listado primeiro.

Um analisador escrito à mão não tem essas regras prontas: elas precisam estar no código. As três subseções mostram onde.

3.1 a) Prioridade de reconhecimento

Toda palavra reservada casa também com o padrão `ID`: `padoinquanto` é uma letra seguida de letras. Aho et al. (2008, seção 3.4.2) descrevem duas maneiras de tratar isso: construir um diagrama de transições para cada palavra reservada, com prioridade sobre o de identificador, ou reconhecer tudo como identificador e depois consultar uma tabela de palavras reservadas. O analisador da Padolang usa a segunda:

1. o autômato de identificador consome o lexema inteiro, enquanto vierem letras ou dígitos;
2. só então o lexema completo é procurado na tabela `RESERVADAS`. Se estiver lá, o token é o da palavra reservada; senão, é `ID`.

```
34      /** Palavras reservadas: o identificador é lido inteiro (maior casamento) e só depois
consultado aqui. */
35      private static final Map<String, TokenType> RESERVADAS = Map.ofEntries(
36              Map.entry("PADOIN", TokenType.PROGRAMA),
37              Map.entry("padoin", TokenType.DECLARA),
38              Map.entry("padoINTER", TokenType.TIPO_INTER),
39              Map.entry("padoFLOAT", TokenType.TIPO_FLOAT),
40              Map.entry("padoOLEAN", TokenType.TIPO_OLEAN),
41              Map.entry("padoSTRING", TokenType.TIPO_STRING),
42              Map.entry("padoIN", TokenType.LEITURA),
43              Map.entry("padoPRINT", TokenType.ESCRITA),
44              Map.entry("padoPRINTLN", TokenType.ESCRITA_LINHA),
45              Map.entry("pad", TokenType.SE),
46              Map.entry("oin", TokenType.SENAO),
47              Map.entry("papapadoin", TokenType.REPETICAO),
48              Map.entry("padoinquanto", TokenType.ENQUANTO),
49              Map.entry("padBREAK", TokenType.INTERROMPE),
50              Map.entry("padCONTINUE", TokenType.PROSSEGUE),
51              Map.entry("padUNCTION", TokenType.FUNCAO),
52              Map.entry("padOUT", TokenType.RETORNO),
53              Map.entry("padoTRUE", TokenType.VERDADEIRO),
54              Map.entry("padoFALSE", TokenType.FALSO),
55              Map.entry("padAND", TokenType.E),
56              Map.entry("padOR", TokenType.OU),
57              Map.entry("padNOT", TokenType.NAO),
58              Map.entry("padoMOD", TokenType.MODULO)
59      );
```

```
156      /** [a-zA-Z_][a-zA-Z0-9_]*  — depois consulta a tabela de reservadas. */
157      private void identificadorOuReservada(int l, int col) {
158          StringBuilder sb = new StringBuilder();
159          while (!fim() && (ehLetra(atual()) || ehDigito(atual()))) {
160              sb.appendCodePoint(avancar());
161          }
162          String lexema = sb.toString();
```

```
163         emitir(RESERVADAS.getOrDefault(lexema, TokenType.ID), lexema, l, col);
164     }
```

A consulta é feita com o lexema **inteiro**, e isso aplica as duas regras na ordem certa. Primeiro o maior casamento: `padoinquanto` é lido até o fim antes de qualquer decisão, então nunca é quebrado em `padoin` + `quanto`. Depois a prioridade: quando o lexema é exatamente uma palavra reservada, ela vence o identificador. Um lexema que só começa como palavra reservada não está na tabela e continua sendo identificador.

Entrada	Token	Por quê
`padoin`	`DECLARA`	o lexema inteiro está na tabela
`padoinquanto`	`ENQUANTO`	lido até o fim; não é `DECLARA` seguido de `ID`
`padoinho`	`ID`	não está na tabela, embora comece por `padoin`
`pad_2`, `oinX`	`ID`	idem, para `pad` e `oin`
`PADOIN`, `padoin`, `padoIN`	`PROGRAMA`, `DECLARA`, `LEITURA`	a comparação diferencia maiúsculas de minúsculas

A tabela tem duas vantagens sobre um diagrama por palavra: o autômato fica com um único caminho para letras, em vez de dezenas de estados para 23 palavras que compartilham os prefixos `pad` e `pado`, e incluir uma palavra reservada nova é acrescentar uma linha. Na especificação em JFlex, o mesmo efeito vem da ordem das regras: as 23 palavras reservadas estão escritas antes da regra `{ID}`, e no empate de tamanho vale a primeira.

3.2 b) Regra do maior casamento

Pela regra do maior casamento (*longest match*), o analisador não emite um token assim que encontra um lexema válido: ele continua enquanto o autômato tiver transição para o próximo símbolo e fica com o último estado final alcançado. No `Lexer.java` isso é feito com **um símbolo de *lookahead***: o método `atual()` examina o próximo símbolo sem consumi-lo, e `casar(esperado)` o consome só quando ele prolonga o lexema.

```
326     /** Consome o próximo símbolo se for o esperado (usado no maior casamento). */
327     private boolean casar(int esperado) {
328         if (!fim() && atual() == esperado) {
329             avancar();
330             return true;
331         }
332         return false;
333     }
```

Para os operadores, o autômato é o da Figura 3. Depois de ler `>`, o analisador está num estado final (`MAIOR`), mas antes de emitir o token verifica se o próximo símbolo é `=`; se for, consome-o e emite `MAIOR_IGUAL`. O operador composto sai como um único token, e nunca como `MAIOR` seguido de `ATRIB`.

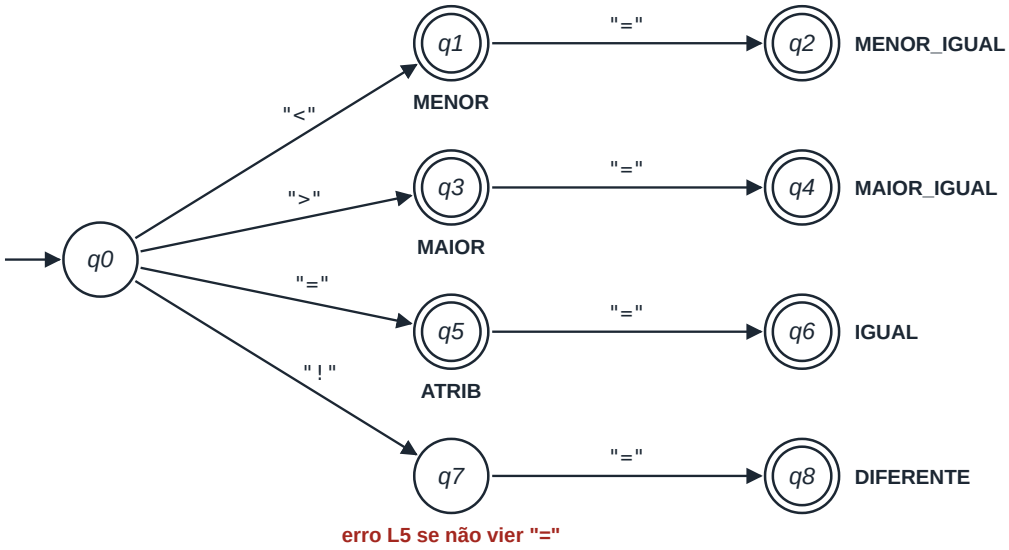


Figura 3 — Diagrama de transições da atribuição e dos operadores relacionais.

```
261      /** Operadores com prefixo comum resolvidos por maior casamento: ++ antes de +, == antes de
=, etc. */
262      private void operadorOuDelimitador(int l, int col) {
263          int c = avancar();
264          switch (c) {
265              case '+' -> emitir(casar('+') ? TokenType.INCREMENTO : TokenType.MAIS, l, col);
266              case '-' -> emitir(casar('-') ? TokenType.DECREMENTO : TokenType.MENOS, l, col);
267              case '*' -> emitir(TokenType.VEZES, l, col);
268              case '/' -> emitir(TokenType.DIVIDIDO, l, col);
269              case '=' -> emitir(casar('=') ? TokenType.IGUAL : TokenType.ATRIB, l, col);
270              case '<' -> emitir(casar('=') ? TokenType.MENOR_IGUAL : TokenType.MENOR, l, col);
271              case '>' -> emitir(casar('=') ? TokenType.MAIOR_IGUAL : TokenType.MAIOR, l, col);
272              case '!' -> {
273                  if (casar('=')) {
274                      emitir(TokenType.DIFERENTE, l, col);
275                  } else {
276                      falha("L5", "'!' isolado não é um operador válido (use !=)", l, col, 1);
277                  }
278              }
279              case '(' -> emitir(TokenType.ABRE_PAR, l, col);
280              case ')' -> emitir(TokenType.FECHA_PAR, l, col);
281              case '{' -> emitir(TokenType.ABRE_CHAVE, l, col);
282              case '}' -> emitir(TokenType.FECHA_CHAVE, l, col);
283              case ',' -> emitir(TokenType.VIRGULA, l, col);
284              default -> falha("L6",
285                      "caractere inválido '" + Character.toString(c) + "' (U+" +
String.format("%04X", c) + ")", l, col, 1);
286          }
287      }
```

A mesma regra aparece nos outros padrões que têm um prefixo em comum:

Padrões em conflito	Como o código decide
`+` e `++`, `-` e `--`, `=` e `==`, `<` e `<=`, `>` e `>=`	`casar()` consome o segundo símbolo, se ele for o esperado
identificadores e palavras reservadas de tamanhos diferentes	o laço de `identificadorOuReservada()` só para quando o próximo símbolo não é letra nem dígito
`NUM_INT` e `NUM_FLOAT`	depois dos dígitos, um `.` prolonga o lexema para a parte fracionária (Figura 1)
comentário de linha e de bloco	depois do `#`, o símbolo `*` leva ao comentário de bloco (Figura 2)

Os exemplos a seguir foram executados no analisador:

Entrada	Tokens produzidos	Observação
`a>=b`	`ID MAIOR_IGUAL ID`	`>=` é um token só
`a> =b`	`ID MAIOR ATRIB ID`	o espaço separa os lexemas
`x===y`	`ID IGUAL ATRIB ID`	o maior casamento vale da esquerda para a direita
`a+++b`	`ID INCREMENTO MAIS ID`	como em C e Java
`5--3`	`NUM_INT DECREMENTO NUM_INT`	erro sintático depois; escreve-se `5 - -3`
`3.14`	`NUM_FLOAT`	não é `NUM_INT`, ponto, `NUM_INT`

O enunciado cita `:=` como exemplo de operador composto. A Padolang não tem esse operador: a atribuição é `=`, e o par equivalente na linguagem é `=` e `==`, tratado como descrito acima. Em `a := 5`, o analisador acusa o `:` como símbolo inválido e reconhece o `=` normalmente (Seção 5.2).

O `5--3` mostra um custo conhecido do maior casamento: a regra é aplicada sem olhar o contexto sintático, e a leitura mais longa nem sempre é a que o programador queria. A Padolang mantém o comportamento de C e Java, e a limitação está documentada no repositório.

Com um símbolo de *lookahead* o analisador nunca precisa devolver símbolos já consumidos. O único ponto em que um analisador gerado recuaria é `3.` seguido de algo que não é dígito, voltando ao estado final q_1 para emitir `NUM_INT`. Como o ponto sozinho não é um token da Padolang, recuar só adiaria o erro; o analisador acusa ali mesmo o número malformado, com o lexema `3.` inteiro.

3.3 c) Controle de posição

O analisador mantém três contadores: `pos`, o índice do próximo símbolo no vetor de entrada; `linha` e `coluna`, a posição desse símbolo no fonte, ambas a partir de 1. O método `avancar()` é o **único ponto do código que consome um símbolo**, e é nele que a posição é atualizada: uma quebra de linha incrementa `linha` e volta `coluna` para 1; qualquer outro símbolo incrementa `coluna`.

```
315     private int avancar() {
316         int c = cp[pos++];
317         if (c == '\n') {
318             linha++;
319             coluna = 1;
320         } else {
321             coluna++;
322         }
323         return c;
324     }
```

Como todos os métodos de reconhecimento consomem símbolos somente por `avancar()` (ou por `casar()`, que o chama), a posição nunca fica defasada, inclusive dentro de brancos, textos e comentários, que podem conter quebras de linha.

A posição de um token é a do **primeiro símbolo do lexema**. O laço principal (Seção 4.4) guarda `linha` e `coluna` em variáveis locais antes de consumir o primeiro símbolo, e passa esses valores ao método que reconhece o lexema. Cada token é um registro com classe, lexema, linha, coluna e tamanho:

```
3  /**
4   * Um token reconhecido: categoria, texto (lexema), posição no fonte e tamanho.
5   *
6   * O tamanho conta code points no fonte original, que pode diferir do lexema: o 👍 com tom de
pele
7   * (👍🏽) tem lexema "👍" mas ocupa 2 code points. O EOF tem tamanho 0.
8   */
9  public record Token(TokenType tipo, String lexema, int linha, int coluna, int tamanho) {
```

Os erros léxicos usam o mesmo mecanismo. Cada erro vira um `Diagnostico` com código, linha, coluna, tamanho do trecho e mensagem, exibido no formato `padoERRO léxico (linha L, coluna C): mensagem`. Em dois casos a posição apontada não é o início do lexema, porque outra é mais útil para quem vai corrigir: uma sequência de escape inválida é apontada na barra invertida, dentro do texto, e um comentário de bloco não fechado é apontado na abertura `#*`, e não no fim do arquivo, onde a falta foi detectada.

Detalhes da contagem:

- **A coluna conta *code points*.** O fonte é convertido para um vetor de *code points* antes da varredura (`fonte.codePoints().toArray()`). Em Java um 👍 ocupa dois `char`; com o vetor, ele ocupa uma posição e conta uma coluna, e a coluna informada coincide com o que o programador vê.
- **Só `\n` muda de linha.** Num arquivo com fim de linha do Windows (`\r\n`), o `\r` é um branco comum no fim da linha e não altera a numeração.
- **A tabulação conta uma coluna**, como qualquer outro símbolo.
- **A marca de ordem de bytes** (U+FEFF), que alguns editores gravam no início do arquivo, é ignorada sem contar coluna.
- **Lexemas de várias linhas** (comentário de bloco) têm a posição da abertura, e a linha continua sendo contada dentro deles. No teste da Seção 5.1 o primeiro comentário ocupa as linhas 1 a 4, e o token seguinte está corretamente na linha 6.

Essa posição acompanha o token pelo resto do compilador: os erros sintáticos e semânticos usam a linha e a coluna do token em que o problema foi encontrado. No JFlex, o equivalente são as diretivas `%line` e `%column`, que mantêm `yyline` e `yycolumn`. Na conferência da Seção 5.4, a única adaptação necessária na especificação foi na coluna: o `yycolumn` do JFlex 1.9.1 conta unidades UTF-16, e portanto duas colunas para cada 👍.

4 Problema 3 — Implementação do analisador léxico

4.1 Organização do código

Arquivo	Conteúdo
`src/padolang/lexer/Lexer.java`	o analisador: laço principal e um método por padrão (Apêndice A)
`src/padolang/lexer/Token.java`	o registro de um token: classe, lexema, linha, coluna e tamanho
`src/padolang/lexer/TokenType.java`	a enumeração das classes de token
`src/padolang/lexer/LexicoException.java`	a exceção de erro léxico do modo compilador
`src/padolang/diagnostico/Diagnostico.java`	o registro de um erro: fase, código, linha, coluna, tamanho e mensagem
`src/padolang/cli/Cli.java`	a linha de comando; o subcomando `tokens` lê o arquivo e imprime a listagem

4.2 Como compilar e executar

É preciso o JDK 21. O analisador recebe o caminho de um arquivo de texto com o programa-fonte e escreve a listagem na saída padrão; para gravá-la em arquivo, basta redirecionar a saída.

```
$ ./compilar.sh
$ bin/padolang tokens --editor programa.padoin                # listagem na saída padrão
$ bin/padolang tokens --editor programa.padoin > saida.tokens   # listagem em arquivo
```

A listagem tem uma linha por token, com cinco colunas: `LIN` e `COL`, a posição do primeiro símbolo; `TAM`, o tamanho do lexema no fonte; `TIPO`, a classe do token; e `LEXEMA`. Depois dos tokens vem uma linha para cada erro léxico, com o código do erro, a linha, a coluna, a mensagem e o lexema apontado. O comando termina com código de saída 1 se houver algum erro léxico, e 0 se não houver.

O analisador tem dois modos, escolhidos no construtor:

- **modo editor** (opção `--editor`, usado neste relatório): os comentários viram tokens `COMENTARIO`, cada erro é registrado e vira um token `ERRO`, e a análise **continua** até o fim do arquivo. É também o modo usado pelo formatador de código e pelo servidor de linguagem dos editores;
- **modo compilador** (padrão, usado para gerar código): os comentários são descartados e o primeiro erro léxico interrompe a compilação com uma exceção, porque as fases seguintes não têm o que fazer com um programa que contém um trecho inválido.

Os tokens válidos são os mesmos nos dois modos.

4.3 Leitura do arquivo

O subcomando `tokens` lê o arquivo inteiro em UTF-8 (`Files.readString`) e entrega o texto ao construtor do analisador, que o converte no vetor de *code points* `cp`:

```
74      public Lexer(String fonte, Modo modo) {
75          this.cp = fonte.codePoints().toArray();
76          this.modo = modo;
77          // Um BOM no início do arquivo não faz parte do programa: é ignorado sem contar coluna
78          if (cp.length > 0 && cp[0] == BOM) {
```

```
79              pos = 1;
80          }
81      }
```

Aho et al. (2008, seção 3.2) descrevem a leitura por pares de *buffers* com sentinelas, pensada para arquivos grandes e pouca memória. Os programas da Padolang são pequenos, então o fonte fica inteiro na memória: a varredura é um índice (`pos`) sobre o vetor, o *lookahead* é uma leitura de `cp[pos]`, e o fim da entrada é o teste `pos >= cp.length` do método `fim()`.

4.4 Varredura

Toda expressão regular denota uma linguagem regular, e toda linguagem regular é reconhecida por um autômato finito determinístico (MENEZES, 2011). O `Lexer.java` é a codificação direta dos autômatos da Seção 2, no estilo dos diagramas de transição de Aho et al. (2008, seção 3.4): o estado é o ponto do código em que a execução está, e cada transição é um teste sobre o símbolo atual seguido de `avancar()`.

O método `tokenizar()` é o estado inicial. Ele descarta os brancos, guarda a posição de início do lexema e escolhe o caminho pelo primeiro símbolo:

```
83      public List<Token> tokenizar() {
84          while (!fim()) {
85              int c = atual();
86
87              if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
88                  avancar();
89                  continue;
90              }
91
92              int l = linha, col = coluna; // posição de início do lexema
93              inicio = pos;
94
95              if (c == '#') {
96                  comentario(l, col);
97              } else if (ehLetra(c)) {
98                  identificadorOuReservada(l, col);
99              } else if (ehDigito(c)) {
100                 numero(l, col);
101             } else if (c == '"') {
102                 string(l, col);
103             } else if (c == JOINHA) {
104                 joinha(l, col);
105             } else {
106                 operadorOuDelimitador(l, col);
107             }
108         }
109         tokens.add(new Token(TokenType.EOF, "<fim>", linha, coluna, 0));
110         return tokens;
111     }
```

A escolha pelo primeiro símbolo é determinística porque os padrões da Seção 2 têm conjuntos de primeiros símbolos disjuntos:

Primeiro símbolo	Método	Tokens possíveis
espaço, `\t`, `\r`, `\n`	(descartado no laço)	nenhum
`#`	`comentario()`	`COMENTARIO`, ou nenhum no modo compilador
letra ou `_`	`identificadorOuReservada()`	`ID` ou uma das 23 palavras reservadas
dígito	`numero()`	`NUM_INT`, `NUM_FLOAT`
`"`	`string()`	`STRING`
👍	`joinha()`	`JOINHA`
qualquer outro	`operadorOuDelimitador()`	operadores e delimitadores, ou erro

Cada método corresponde a um diagrama. O dos números implementa a Figura 1, com as setas tracejadas de erro:

```
166      /** [0-9]+ (NUM_INT)  ou  [0-9]+\.[0-9]+ (NUM_FLOAT): no máximo um ponto, com dígitos dos
dois lados. */
167      private void numero(int l, int col) {
168          StringBuilder sb = new StringBuilder();
169          lerDigitos(sb);
170
171          TokenType tipo = TokenType.NUM_INT;
172          if (!fim() && atual() == '.') {
173              sb.appendCodePoint(avancar());
174              if (fim() || !ehDigito(atual())) {
175                  falha("L1", "número real malformado '" + sb + "' (esperado dígito após o
ponto)", l, col, pos - inicio);
176                  return;
177              }
178              lerDigitos(sb);
179              tipo = TokenType.NUM_FLOAT;
180
181              if (!fim() && atual() == '.') {
182                  // 1.2.3: o lexema inteiro é um erro só, em vez de um real seguido de um '.'
solto
183                  while (!fim() && (ehDigito(atual()) || atual() == '.')) {
184                      avancar();
185                  }
186                  falha("L1", "número real malformado '" + trecho() + "' (mais de um ponto
decimal)", l, col, pos - inicio);
187                  return;
188              }
189          }
190
191          if (!fim() && ehLetra(atual())) {
192              // O trecho apontado vai até o fim do "identificador" (ex.: 2x inteiro)
193              int fimTrecho = pos;
194              while (fimTrecho < cp.length && (ehLetra(cp[fimTrecho]) ||
ehDigito(cp[fimTrecho]))) {
195                  fimTrecho++;
196              }
197              falhaSemToken("L2", "lexema inválido '" + new String(cp, inicio, fimTrecho -
inicio)
198                      + "' (identificadores não podem começar com dígito)", l, col, fimTrecho -
inicio);
199              while (pos < fimTrecho) {
200                  avancar();
201              }
202              emitir(TokenType.ERRO, trecho(), l, col);
203              return;
204          }
205          emitir(tipo, sb.toString(), l, col);
206      }
```

O dos comentários implementa a Figura 2. O `if` depois do `#` é o símbolo de *lookahead* que separa os dois tipos, e o laço de `comentarioDeBloco()` percorre os estados q_3 e q_4 até encontrar um `*` seguido de `#`:

```
127     private void comentario(int l, int col) {
128         avancar(); // '#'
129         if (!fim() && atual() == '*') {
130             comentarioDeBloco(l, col);
131             return;
132         }
133         while (!fim() && atual() != '\n') {
134             avancar();
135         }
136         if (modo == Modo.EDITOR) {
137             emitir(TokenType.COMENTARIO, trecho(), l, col);
138         }
139     }
```

```
141     /** O resto do comentário de bloco, depois do '#'. Não aninha: fecha no primeiro "*#". */
142     private void comentarioDeBloco(int l, int col) {
143         avancar(); // '*'
144         while (!fim()) {
145             if (avancar() == '*' && casar('#')) {
146                 if (modo == Modo.EDITOR) {
147                     emitir(TokenType.COMENTARIO, trecho(), l, col);
148                 }
149                 return;
150             }
151         }
152         // O erro aponta o "#*" que ficou aberto; no modo EDITOR o trecho até o fim do arquivo
vira ERRO
153         falha("L7", "comentário de bloco não fechado (falta o '*#')", l, col, 2);
154     }
```

Os métodos de identificadores e de operadores estão na Seção 3. O analisador completo está no Apêndice A.

4.5 Tratamento de erros

Há sete erros léxicos, identificados por um código. Os dois primeiros e o quinto correspondem às setas tracejadas das Figuras 1 e 3; o último, à nota da Figura 2.

Código	Situação	Exemplo	Trecho descartado
`L1`	número real malformado: ponto sem dígito depois, ou mais de um ponto	`3.` `1.2.3`	os dígitos e os pontos do número
`L2`	número seguido de letra (identificador iniciado por dígito)	`2x` `123abc`	toda a sequência de letras e dígitos
`L3`	texto sem a aspa de fechamento	`"aberto`	da aspa até o fim da linha
`L4`	sequência de escape inexistente num texto	`"a\qb"`	o texto inteiro, até a aspa de fechamento
`L5`	`!` que não é seguido de `=`	`!a`	o `!`
`L6`	símbolo que não pertence ao alfabeto	`@` `;` `ç`	o símbolo
`L7`	comentário de bloco sem `*#`	`#* aberto`	da abertura até o fim do arquivo

Todos os erros passam por dois métodos. No modo compilador, `falhaSemToken()` lança a exceção e a análise termina. No modo editor, o diagnóstico vai para a lista `erros`, o trecho inválido é emitido como um token `ERRO` e o método retorna; o laço principal segue então para o próximo lexema.

```
295     private void falha(String codigo, String mensagem, int l, int col, int tamanho) {
296         falhaSemToken(codigo, mensagem, l, col, tamanho);
297         emitir(TokenType.ERRO, trecho(), l, col);
298     }
```

```
301     private void falhaSemToken(String codigo, String mensagem, int l, int col, int tamanho) {
302         Diagnostico d = Diagnostico.erro(Fase.LEXICO, codigo, l, col, tamanho, mensagem);
303         if (modo == Modo.COMPILADOR) {
304             throw new LexicoException(d);
305         }
306         erros.add(d);
307     }
```

A recuperação segue a estratégia de *modo pânico* de Aho et al. (2008, seção 3.1.4): descartam-se símbolos da entrada até que o analisador possa reconhecer um token bem formado. O que muda de um erro para outro é **quanto descartar** (última coluna da tabela), e a escolha foi feita para que um único engano do programador gere uma única mensagem:

- num símbolo inválido, descarta-se só ele: em `1 @ 2`, os dois números continuam sendo reconhecidos;
- em `2x`, descarta-se a palavra toda. Se só o `2` fosse descartado, o `x` restante seria aceito como um identificador que o programador não escreveu;
- em `1.2.3`, descarta-se o número todo, em vez de aceitar `1.2` e acusar em seguida um ponto sem sentido;
- num texto não fechado, descarta-se até o fim da linha. Como um texto não pode conter quebra de linha, a linha seguinte começa num estado conhecido e é analisada normalmente;
- num comentário de bloco não fechado não há ponto seguro para retomar, porque todo o resto do arquivo pode ser comentário. O trecho vai até o fim, como nos compiladores de C e Java.

A listagem é impressa por `tokensEditor()`, na classe `Cli`. O lexema de cada erro é recortado do próprio fonte, pela linha, coluna e tamanho do diagnóstico:

```
356     private int tokensEditor(String fonte) {
357         Lexer lexer = new Lexer(fonte, Lexer.Modo.EDITOR);
358         List<Token> tokens = lexer.tokenizar();
359         out.println("== TOKENS (editor) ==");
360         out.printf("%-5s %-5s %-4s %-13s %s%n", "LIN", "COL", "TAM", "TIPO", "LEXEMA");
361         for (Token t : tokens) {
362             out.printf("%-5d %-5d %-4d %-13s %s%n", t.linha(), t.coluna(), t.tamanho(),
t.tipo(), t.lexemaEmUmaLinha());
363         }
364         for (Diagnostico d : lexer.erros()) {
365             out.println(d.codigo() + " " + d.formatar() + " [lexema: " + trecho(fonte, d) +
"]");
366         }
367         return lexer.erros().isEmpty() ? OK : ERRO_PROGRAMA;
368     }
```

5 Problema 4 — Casos de teste e evidências de validação

Os testes do analisador ficam em `testes/lexer_editor/`. Cada caso é um par de arquivos: a entrada `nome.padoin` e a saída esperada `nome.tokens`. As listagens desta seção são o conteúdo desses arquivos `.tokens`, gerados pela execução do analisador, e o roteiro da Seção 5.3 confere a cada execução que o analisador continua produzindo exatamente essas saídas.

Arquivo de entrada	Objetivo	Resultado
`todos_os_tokens.padoin`	programa válido com todas as classes de token (item *a* do enunciado)	294 tokens, 48 classes, nenhum erro
`erros_lexicos.padoin`	erros léxicos propositais, dos sete tipos (item *b*)	19 erros, todos com posição e lexema; 97 tokens válidos reconhecidos em volta deles
`comentario_de_bloco.padoin`	casos-limite do comentário de bloco: `#**#`, várias linhas, dentro de texto, fim de linha `\r\n`	sem erros
`comentario_nao_fechado.padoin`	`#*#` não fecha o comentário	erro `L7`
`comentarios`, `cedilha_fora_de_texto`, `texto_nao_fechado`, `todos_os_erros`	casos que já existiam no repositório	conforme o esperado

5.1 Teste com código válido

O arquivo `todos_os_tokens.padoin` é um programa correto: além de passar pelo analisador léxico, passa pelas análises sintática e semântica e executa. Ele usa as 23 palavras reservadas, identificadores (inclusive com prefixo de palavra reservada), números inteiros e reais, textos com sequências de escape, todos os operadores (com e sem espaços em volta), todos os delimitadores e os dois tipos de comentário.

```
#* Caso de teste válido do analisador léxico.
   O programa está correto e usa todas as classes de token da padolang:
   as 23 palavras reservadas, identificadores, números inteiros e reais,
   textos, todos os operadores, os delimitadores e os dois tipos de comentário. *#

# funções: padUNCTION, os tipos, a vírgula e o padOUT
padUNCTION padoINTER soma(padoINTER a, padoINTER b) {
  padOUT a + b👍
}

padUNCTION padoOLEAN ehPar(padoINTER n) {
  padOUT n padoMOD 2 == 0👍
}

PADOIN {
  # declarações, com um literal de cada tipo
  padoin padoINTER limite👍
  padoin padoINTER i = 0👍
  padoin padoINTER pares = 0👍
  padoin padoFLOAT media = 7.25👍
  padoin padoOLEAN ligado = padoTRUE👍
  padoin padoOLEAN desligado = padoFALSE👍
  padoin padoSTRING nome_1 = "Padoin"👍
  padoin padoSTRING _escapes = "tab:\t aspas:\" barra:\\ # nao e comentario"👍

  # identificadores que começam como palavras reservadas (maior casamento)
  padoin padoINTER padoinho = 1👍
```

```
  padoin padoINTER pad_2 = 2👍
  padoin padoINTER oinX = 3👍

  # entrada e saída
  padoPRINT("Limite: ")👍
  padoIN(limite)👍
  padoPRINTLN()👍

  # operadores aritméticos, com e sem espaços em volta
  media = (media + 2.75) * 3 / 2 - 0.5👍
  padoinho = soma(pad_2, oinX)*2-1👍
  padoinho++👍
  pad_2--👍
  oinX = 5 - -3👍

  # operadores relacionais e lógicos
  pad (limite >= 10 padAND limite <= 100) {
    padoPRINTLN("entre 10 e 100")👍
  } oin pad (limite<10 padOR limite>100) {
    padoPRINTLN("fora da faixa")👍
  } oin {
    padoPRINTLN("impossivel")👍
  }
  pad (padNOT desligado padAND ligado != desligado) {
    padoPRINTLN(nome_1 + _escapes)👍
  }

  # repetições: papapadoin, padoinquanto, padBREAK e padCONTINUE
  papapadoin (i = 1, i < limite, i++) {
    pad (padNOT ehPar(i)) {
      padCONTINUE👍
    }
    pares++👍
  }
  padoinquanto (padoTRUE) {
    pad (pares == 0 padOR pares > 3) {
      padBREAK👍
    }
    pares--👍
  }

  padoPRINTLN(media)👍🏾   #* o 👍 com tom de pele é o mesmo token *#
  padoPRINTLN(padoinho + pad_2 + oinX + pares)👍
}
```

Saída do analisador para `bin/padolang tokens --editor testes/lexer_editor/todos_os_tokens.padoin`:

```
== TOKENS (editor) ==
LIN   COL   TAM  TIPO          LEXEMA
1     1     273  COMENTARIO    #* Caso de teste
válido do analisador léxico.\n   O programa está
correto e usa todas as classes de token da
padolang:\n   as 23 palavras reservadas,
identificadores, números inteiros e reais,\n
textos, todos os operadores, os delimitadores e os
dois tipos de comentário. *#
6     1     53   COMENTARIO    # funções: padUNCTION,
os tipos, a vírgula e o padOUT
7     1     10   FUNCAO        padUNCTION
7     12    9    TIPO_INTER    padoINTER
7     22    4    ID            soma
7     26    1    ABRE_PAR      (
7     27    9    TIPO_INTER    padoINTER
7     37    1    ID            a
7     38    1    VIRGULA       ,
7     40    9    TIPO_INTER    padoINTER
7     50    1    ID            b
7     51    1    FECHA_PAR     )
7     53    1    ABRE_CHAVE    {
8     3     6    RETORNO       padOUT
8     10    1    ID            a
8     12    1    MAIS          +
8     14    1    ID            b
8     15    1    JOINHA        👍
9     1     1    FECHA_CHAVE   }
11    1     10   FUNCAO        padUNCTION
11    12    9    TIPO_OLEAN    padoOLEAN
11    22    5    ID            ehPar
11    27    1    ABRE_PAR      (
11    28    9    TIPO_INTER    padoINTER
11    38    1    ID            n
11    39    1    FECHA_PAR     )
11    41    1    ABRE_CHAVE    {
12    3     6    RETORNO       padOUT
12    10    1    ID            n
12    12    7    MODULO        padoMOD
12    20    1    NUM_INT       2
12    22    2    IGUAL         ==
12    25    1    NUM_INT       0
12    26    1    JOINHA        👍
13    1     1    FECHA_CHAVE   }
15    1     6    PROGRAMA      PADOIN
15    8     1    ABRE_CHAVE    {
16    3     42   COMENTARIO    # declarações, com um
literal de cada tipo
```

```
17    3     6    DECLARA       padoin
17    10    9    TIPO_INTER    padoINTER
17    20    6    ID            limite
17    26    1    JOINHA        👍
18    3     6    DECLARA       padoin
18    10    9    TIPO_INTER    padoINTER
18    20    1    ID            i
18    22    1    ATRIB         =
18    24    1    NUM_INT       0
18    25    1    JOINHA        👍
19    3     6    DECLARA       padoin
19    10    9    TIPO_INTER    padoINTER
19    20    5    ID            pares
19    26    1    ATRIB         =
19    28    1    NUM_INT       0
19    29    1    JOINHA        👍
20    3     6    DECLARA       padoin
20    10    9    TIPO_FLOAT    padoFLOAT
20    20    5    ID            media
20    26    1    ATRIB         =
20    28    4    NUM_FLOAT     7.25
20    32    1    JOINHA        👍
21    3     6    DECLARA       padoin
21    10    9    TIPO_OLEAN    padoOLEAN
21    20    6    ID            ligado
21    27    1    ATRIB         =
21    29    8    VERDADEIRO    padoTRUE
21    37    1    JOINHA        👍
22    3     6    DECLARA       padoin
22    10    9    TIPO_OLEAN    padoOLEAN
22    20    9    ID            desligado
22    30    1    ATRIB         =
22    32    9    FALSO         padoFALSE
22    41    1    JOINHA        👍
23    3     6    DECLARA       padoin
23    10    10   TIPO_STRING   padoSTRING
23    21    6    ID            nome_1
23    28    1    ATRIB         =
23    30    8    STRING        "Padoin"
23    38    1    JOINHA        👍
24    3     6    DECLARA       padoin
24    10    10   TIPO_STRING   padoSTRING
24    21    8    ID            _escapes
24    30    1    ATRIB         =
24    32    45   STRING        "tab:\t aspas:\"
barra:\\ # nao e comentario"
24    77    1    JOINHA        👍
26    3     72   COMENTARIO    # identificadores que
começam como palavras reservadas (maior casamento)
27    3     6    DECLARA       padoin
27    10    9    TIPO_INTER    padoINTER
27    20    8    ID            padoinho
27    29    1    ATRIB         =
27    31    1    NUM_INT       1
27    32    1    JOINHA        👍
28    3     6    DECLARA       padoin
28    10    9    TIPO_INTER    padoINTER
28    20    5    ID            pad_2
28    26    1    ATRIB         =
28    28    1    NUM_INT       2
28    29    1    JOINHA        👍
29    3     6    DECLARA       padoin
29    10    9    TIPO_INTER    padoINTER
29    20    4    ID            oinX
29    25    1    ATRIB         =
29    27    1    NUM_INT       3
29    28    1    JOINHA        👍
31    3     17   COMENTARIO    # entrada e saída
32    3     9    ESCRITA       padoPRINT
32    12    1    ABRE_PAR      (
32    13    10   STRING        "Limite: "
32    23    1    FECHA_PAR     )
32    24    1    JOINHA        👍
33    3     6    LEITURA       padoIN
33    9     1    ABRE_PAR      (
33    10    6    ID            limite
33    16    1    FECHA_PAR     )
33    17    1    JOINHA        👍
34    3     11   ESCRITA_LINHA padoPRINTLN
34    14    1    ABRE_PAR      (
34    15    1    FECHA_PAR     )
34    16    1    JOINHA        👍
36    3     52   COMENTARIO    # operadores
aritméticos, com e sem espaços em volta
37    3     5    ID            media
37    9     1    ATRIB         =
37    11    1    ABRE_PAR      (
37    12    5    ID            media
37    18    1    MAIS          +
37    20    4    NUM_FLOAT     2.75
37    24    1    FECHA_PAR     )
37    26    1    VEZES         *
37    28    1    NUM_INT       3
37    30    1    DIVIDIDO      /
37    32    1    NUM_INT       2
37    34    1    MENOS         -
37    36    3    NUM_FLOAT     0.5
37    39    1    JOINHA        👍
38    3     8    ID            padoinho
38    12    1    ATRIB         =
38    14    4    ID            soma
38    18    1    ABRE_PAR      (
38    19    5    ID            pad_2
38    24    1    VIRGULA       ,
38    26    4    ID            oinX
38    30    1    FECHA_PAR     )
38    31    1    VEZES         *
38    32    1    NUM_INT       2
38    33    1    MENOS         -
38    34    1    NUM_INT       1
38    35    1    JOINHA        👍
39    3     8    ID            padoinho
39    11    2    INCREMENTO    ++
39    13    1    JOINHA        👍
40    3     5    ID            pad_2
40    8     2    DECREMENTO    --
40    10    1    JOINHA        👍
41    3     4    ID            oinX
41    8     1    ATRIB         =
41    10    1    NUM_INT       5
41    12    1    MENOS         -
41    14    1    MENOS         -
41    15    1    NUM_INT       3
41    16    1    JOINHA        👍
43    3     34   COMENTARIO    # operadores
relacionais e lógicos
44    3     3    SE            pad
44    7     1    ABRE_PAR      (
44    8     6    ID            limite
44    15    2    MAIOR_IGUAL   >=
44    18    2    NUM_INT       10
44    21    6    E             padAND
44    28    6    ID            limite
44    35    2    MENOR_IGUAL   <=
44    38    3    NUM_INT       100
44    41    1    FECHA_PAR     )
44    43    1    ABRE_CHAVE    {
45    5     11   ESCRITA_LINHA padoPRINTLN
45    16    1    ABRE_PAR      (
45    17    16   STRING        "entre 10 e 100"
45    33    1    FECHA_PAR     )
45    34    1    JOINHA        👍
46    3     1    FECHA_CHAVE   }
46    5     3    SENAO         oin
46    9     3    SE            pad
46    13    1    ABRE_PAR      (
46    14    6    ID            limite
46    20    1    MENOR         <
46    21    2    NUM_INT       10
46    24    5    OU            padOR
46    30    6    ID            limite
46    36    1    MAIOR         >
46    37    3    NUM_INT       100
46    40    1    FECHA_PAR     )
46    42    1    ABRE_CHAVE    {
47    5     11   ESCRITA_LINHA padoPRINTLN
47    16    1    ABRE_PAR      (
47    17    15   STRING        "fora da faixa"
47    32    1    FECHA_PAR     )
47    33    1    JOINHA        👍
```

```
48    3     1    FECHA_CHAVE   }
48    5     3    SENAO         oin
48    9     1    ABRE_CHAVE    {
49    5     11   ESCRITA_LINHA padoPRINTLN
49    16    1    ABRE_PAR      (
49    17    12   STRING        "impossivel"
49    29    1    FECHA_PAR     )
49    30    1    JOINHA        👍
50    3     1    FECHA_CHAVE   }
51    3     3    SE            pad
51    7     1    ABRE_PAR      (
51    8     6    NAO           padNOT
51    15    9    ID            desligado
51    25    6    E             padAND
51    32    6    ID            ligado
51    39    2    DIFERENTE     !=
51    42    9    ID            desligado
51    51    1    FECHA_PAR     )
51    53    1    ABRE_CHAVE    {
52    5     11   ESCRITA_LINHA padoPRINTLN
52    16    1    ABRE_PAR      (
52    17    6    ID            nome_1
52    24    1    MAIS          +
52    26    8    ID            _escapes
52    34    1    FECHA_PAR     )
52    35    1    JOINHA        👍
53    3     1    FECHA_CHAVE   }
55    3     62   COMENTARIO    # repetições:
papapadoin, padoinquanto, padBREAK e padCONTINUE
56    3     10   REPETICAO     papapadoin
56    14    1    ABRE_PAR      (
56    15    1    ID            i
56    17    1    ATRIB         =
56    19    1    NUM_INT       1
56    20    1    VIRGULA       ,
56    22    1    ID            i
56    24    1    MENOR         <
56    26    6    ID            limite
56    32    1    VIRGULA       ,
56    34    1    ID            i
56    35    2    INCREMENTO    ++
56    37    1    FECHA_PAR     )
56    39    1    ABRE_CHAVE    {
57    5     3    SE            pad
57    9     1    ABRE_PAR      (
57    10    6    NAO           padNOT
57    17    5    ID            ehPar
57    22    1    ABRE_PAR      (
57    23    1    ID            i
57    24    1    FECHA_PAR     )
57    25    1    FECHA_PAR     )
57    27    1    ABRE_CHAVE    {
58    7     11   PROSSEGUE     padCONTINUE
58    18    1    JOINHA        👍
59    5     1    FECHA_CHAVE   }
60    5     5    ID            pares
60    10    2    INCREMENTO    ++
60    12    1    JOINHA        👍
61    3     1    FECHA_CHAVE   }
62    3     12   ENQUANTO      padoinquanto
62    16    1    ABRE_PAR      (
62    17    8    VERDADEIRO    padoTRUE
62    25    1    FECHA_PAR     )
62    27    1    ABRE_CHAVE    {
63    5     3    SE            pad
63    9     1    ABRE_PAR      (
63    10    5    ID            pares
63    16    2    IGUAL         ==
63    19    1    NUM_INT       0
63    21    5    OU            padOR
63    27    5    ID            pares
63    33    1    MAIOR         >
63    35    1    NUM_INT       3
63    36    1    FECHA_PAR     )
63    38    1    ABRE_CHAVE    {
64    7     8    INTERROMPE    padBREAK
64    15    1    JOINHA        👍
65    5     1    FECHA_CHAVE   }
66    5     5    ID            pares
66    10    2    DECREMENTO    --
66    12    1    JOINHA        👍
67    3     1    FECHA_CHAVE   }
69    3     11   ESCRITA_LINHA padoPRINTLN
69    14    1    ABRE_PAR      (
69    15    5    ID            media
69    20    1    FECHA_PAR     )
69    21    2    JOINHA        👍
69    26    41   COMENTARIO    #* o 👍 com tom de
pele é o mesmo token *#
70    3     11   ESCRITA_LINHA padoPRINTLN
70    14    1    ABRE_PAR      (
70    15    8    ID            padoinho
70    24    1    MAIS          +
70    26    5    ID            pad_2
70    32    1    MAIS          +
70    34    4    ID            oinX
70    39    1    MAIS          +
70    41    5    ID            pares
70    46    1    FECHA_PAR     )
70    47    1    JOINHA        👍
71    1     1    FECHA_CHAVE   }
72    1     0    EOF           <fim>
```

Pontos conferidos na listagem:

Linha do fonte	O que se observa	Regra validada
1 a 4	um único token `COMENTARIO` na linha 1, coluna 1, com 273 símbolos; o token seguinte está na linha 6	comentário de bloco; contagem de linhas dentro do lexema
27 a 29	`padoinho`, `pad_2` e `oinX` são `ID`, e o `padoin` da mesma linha é `DECLARA`	maior casamento e prioridade das palavras reservadas
25	o texto com `\t`, `\"`, `\\` e `#` é um único `STRING`	escapes; `#` dentro de texto não é comentário
38	`soma(pad_2, oinX)*2-1`, sem espaços, é separado em 10 tokens	os lexemas não dependem de brancos
39 e 40	`++` e `--` são `INCREMENTO` e `DECREMENTO`, com tamanho 2	maior casamento nos operadores
41	`5 - -3` produz dois `MENOS`, nas colunas 12 e 14	o branco separa os lexemas
44 e 46	`>=` e `<=` são um token cada; em `limite<10` e `limite>100`, `MENOR` e `MAIOR`	maior casamento com um símbolo de *lookahead*
69	o 👍🏽 é `JOINHA` com tamanho 2 e lexema 👍; o comentário depois dele está na coluna 26	modificador de tom de pele; coluna em *code points*
72	`EOF` na linha 72, coluna 1, com tamanho 0	fim de arquivo

O programa também é aceito pelo restante do compilador, o que confirma que os tokens entregues são os que a gramática espera:

```
$ bin/padolang check testes/lexer_editor/todos_os_tokens.padoin
testes/lexer_editor/todos_os_tokens.padoin: sem erros
```

5.2 Teste com erros léxicos

O arquivo `erros_lexicos.padoin` provoca os sete tipos de erro. Os comentários do próprio arquivo dizem o que cada bloco testa, e são reconhecidos normalmente como tokens `COMENTARIO`.

```
# Caso de teste com erros léxicos propositais.
# Cada bloco provoca um tipo de erro; os tokens válidos em volta continuam sendo reconhecidos.
PADOIN {
  # L6: símbolos que não pertencem ao alfabeto da linguagem
  padoin padoINTER a = 1 @ 2👍
  padoin padoINTER b = a $ 3; b = b % 2👍
  padoin padoINTER ação = 4👍

  # L6: operadores de outras linguagens (":=", "&&") não existem na padolang
  a := 5👍
  pad (a > 1 && b < 2) { }

  # L2: identificador iniciado por dígito
  padoin padoINTER 2x = 6👍
  padoin padoFLOAT 9.5nota = 123abc👍

  # L1: constantes numéricas malformadas
  padoin padoFLOAT c = 3.👍
  padoin padoFLOAT d = 1.2.3👍
  padoin padoFLOAT e = .5👍

  # L5: "!" sozinho (só existe o "!=")
  pad (!a != b) { }

  # L4: sequência de escape que não existe
  padoPRINTLN("um\qdois")👍

  # L3: texto sem a aspa de fechamento
  padoPRINTLN("aberto)👍
  padoPRINTLN("depois do erro, tudo normal")👍
}

#* L7: comentário de bloco sem fechamento;
   o erro aponta a linha e a coluna onde ele foi aberto
```

Saída do analisador para `bin/padolang tokens --editor testes/lexer_editor/erros_lexicos.padoin`. Primeiro os tokens; os trechos inválidos aparecem na posição em que ocorrem, com a classe `ERRO`:

```
== TOKENS (editor) ==
LIN   COL   TAM  TIPO          LEXEMA
1     1     46   COMENTARIO    # Caso de teste com
erros léxicos propositais.
2     1     94   COMENTARIO    # Cada bloco provoca
um tipo de erro; os tokens válidos em volta continuam
sendo reconhecidos.
3     1     6    PROGRAMA      PADOIN
3     8     1    ABRE_CHAVE    {
4     3     57   COMENTARIO    # L6: símbolos que não
pertencem ao alfabeto da linguagem
5     3     6    DECLARA       padoin
5     10    9    TIPO_INTER    padoINTER
5     20    1    ID            a
5     22    1    ATRIB         =
5     24    1    NUM_INT       1
5     26    1    ERRO          @
5     28    1    NUM_INT       2
5     29    1    JOINHA        👍
6     3     6    DECLARA       padoin
6     10    9    TIPO_INTER    padoINTER
6     20    1    ID            b
6     22    1    ATRIB         =
6     24    1    ID            a
6     26    1    ERRO          $
6     28    1    NUM_INT       3
6     29    1    ERRO          ;
6     31    1    ID            b
6     33    1    ATRIB         =
6     35    1    ID            b
6     37    1    ERRO          %
6     39    1    NUM_INT       2
6     40    1    JOINHA        👍
7     3     6    DECLARA       padoin
7     10    9    TIPO_INTER    padoINTER
7     20    1    ID            a
7     21    1    ERRO          ç
7     22    1    ERRO          ã
7     23    1    ID            o
7     25    1    ATRIB         =
7     27    1    NUM_INT       4
7     28    1    JOINHA        👍
9     3     74   COMENTARIO    # L6: operadores de
outras linguagens (":=", "&&") não existem na
padolang
10    3     1    ID            a
10    5     1    ERRO          :
10    6     1    ATRIB         =
```

```
10    8     1    NUM_INT       5
10    9     1    JOINHA        👍
11    3     3    SE            pad
11    7     1    ABRE_PAR      (
11    8     1    ID            a
11    10    1    MAIOR         >
11    12    1    NUM_INT       1
11    14    1    ERRO          &
11    15    1    ERRO          &
11    17    1    ID            b
11    19    1    MENOR         <
11    21    1    NUM_INT       2
11    22    1    FECHA_PAR     )
11    24    1    ABRE_CHAVE    {
11    26    1    FECHA_CHAVE   }
13    3     39   COMENTARIO    # L2: identificador
iniciado por dígito
14    3     6    DECLARA       padoin
14    10    9    TIPO_INTER    padoINTER
14    20    2    ERRO          2x
14    23    1    ATRIB         =
14    25    1    NUM_INT       6
14    26    1    JOINHA        👍
15    3     6    DECLARA       padoin
15    10    9    TIPO_FLOAT    padoFLOAT
15    20    7    ERRO          9.5nota
15    28    1    ATRIB         =
15    30    6    ERRO          123abc
15    36    1    JOINHA        👍
17    3     38   COMENTARIO    # L1: constantes
numéricas malformadas
18    3     6    DECLARA       padoin
18    10    9    TIPO_FLOAT    padoFLOAT
18    20    1    ID            c
18    22    1    ATRIB         =
18    24    2    ERRO          3.
18    26    1    JOINHA        👍
19    3     6    DECLARA       padoin
19    10    9    TIPO_FLOAT    padoFLOAT
19    20    1    ID            d
19    22    1    ATRIB         =
19    24    5    ERRO          1.2.3
19    29    1    JOINHA        👍
20    3     6    DECLARA       padoin
20    10    9    TIPO_FLOAT    padoFLOAT
20    20    1    ID            e
20    22    1    ATRIB         =
20    24    1    ERRO          .
20    25    1    NUM_INT       5
20    26    1    JOINHA        👍
22    3     36   COMENTARIO    # L5: "!" sozinho (só
existe o "!=")
23    3     3    SE            pad
23    7     1    ABRE_PAR      (
23    8     1    ERRO          !
23    9     1    ID            a
23    11    2    DIFERENTE     !=
23    14    1    ID            b
23    15    1    FECHA_PAR     )
23    17    1    ABRE_CHAVE    {
23    19    1    FECHA_CHAVE   }
25    3     40   COMENTARIO    # L4: sequência de
escape que não existe
26    3     11   ESCRITA_LINHA padoPRINTLN
26    14    1    ABRE_PAR      (
26    15    10   ERRO          "um\qdois"
26    25    1    FECHA_PAR     )
26    26    1    JOINHA        👍
28    3     36   COMENTARIO    # L3: texto sem a aspa
de fechamento
29    3     11   ESCRITA_LINHA padoPRINTLN
29    14    1    ABRE_PAR      (
29    15    9    ERRO          "aberto)👍
30    3     11   ESCRITA_LINHA padoPRINTLN
30    14    1    ABRE_PAR      (
30    15    29   STRING        "depois do erro, tudo
normal"
30    44    1    FECHA_PAR     )
30    45    1    JOINHA        👍
31    1     1    FECHA_CHAVE   }
33    1     99   ERRO          #* L7: comentário de
bloco sem fechamento;\n   o erro aponta a linha e a
coluna onde ele foi aberto\n
35    1     0    EOF           <fim>
```

Em seguida, na mesma saída, os 19 erros. Cada linha traz o código, a linha e a coluna, a descrição e o lexema problemático:

```
L6 padoERRO léxico (linha 5, coluna 26): caractere inválido '@' (U+0040) [lexema: @]
L6 padoERRO léxico (linha 6, coluna 26): caractere inválido '$' (U+0024) [lexema: $]
L6 padoERRO léxico (linha 6, coluna 29): caractere inválido ';' (U+003B) [lexema: ;]
L6 padoERRO léxico (linha 6, coluna 37): caractere inválido '%' (U+0025) [lexema: %]
L6 padoERRO léxico (linha 7, coluna 21): caractere inválido 'ç' (U+00E7) [lexema: ç]
L6 padoERRO léxico (linha 7, coluna 22): caractere inválido 'ã' (U+00E3) [lexema: ã]
L6 padoERRO léxico (linha 10, coluna 5): caractere inválido ':' (U+003A) [lexema: :]
L6 padoERRO léxico (linha 11, coluna 14): caractere inválido '&' (U+0026) [lexema: &]
L6 padoERRO léxico (linha 11, coluna 15): caractere inválido '&' (U+0026) [lexema: &]
L2 padoERRO léxico (linha 14, coluna 20): lexema inválido '2x' (identificadores não podem começar com dígito)
[lexema: 2x]
L2 padoERRO léxico (linha 15, coluna 20): lexema inválido '9.5nota' (identificadores não podem começar com
dígito) [lexema: 9.5nota]
L2 padoERRO léxico (linha 15, coluna 30): lexema inválido '123abc' (identificadores não podem começar com
dígito) [lexema: 123abc]
L1 padoERRO léxico (linha 18, coluna 24): número real malformado '3.' (esperado dígito após o ponto) [lexema:
3.]
L1 padoERRO léxico (linha 19, coluna 24): número real malformado '1.2.3' (mais de um ponto decimal) [lexema:
1.2.3]
L6 padoERRO léxico (linha 20, coluna 24): caractere inválido '.' (U+002E) [lexema: .]
L5 padoERRO léxico (linha 23, coluna 8): '!' isolado não é um operador válido (use !=) [lexema: !]
L4 padoERRO léxico (linha 26, coluna 18): sequência de escape inválida na string [lexema: \q]
L3 padoERRO léxico (linha 29, coluna 15): string não fechada [lexema: "aberto)👍]
L7 padoERRO léxico (linha 33, coluna 1): comentário de bloco não fechado (falta o '*#') [lexema: #*]
```

Conferência dos erros esperados contra os relatados:

Linha do fonte	Erro provocado	Relatado	Recuperação observada
5	`@` entre dois números	`L6`, coluna 26	`1`, `2` e 👍 reconhecidos em volta
6	`$`, `;` e `%` na mesma linha	três `L6`, colunas 26, 29 e 37	vários erros na mesma linha, cada um com a sua coluna
7	`ação` como identificador	`L6` para `ç` e para `ã`, colunas 21 e 22	`a` e `o` saem como `ID`
10	`:=`	`L6` para `:`, coluna 5	o `=` é reconhecido como `ATRIB`
11	`&&`	dois `L6`, colunas 14 e 15	o restante da condição é reconhecido
14	`2x`	`L2`, coluna 20, lexema `2x`	um erro só; não sobra um `ID x`
15	`9.5nota` e `123abc`	`L2` nas colunas 20 e 30	lexema inteiro em cada erro
18	`3.`	`L1`, coluna 24	o 👍 seguinte é reconhecido
19	`1.2.3`	`L1`, coluna 24, “mais de um ponto decimal”	um erro só, com o lexema inteiro
20	`.5`	`L6` para o `.`, coluna 24	o `5` é reconhecido como `NUM_INT`
23	`!` sozinho	`L5`, coluna 8	o `!=` da mesma linha é `DIFERENTE`
26	escape `\q`	`L4`, coluna 18 (a da barra), lexema `\q`	o texto vira um `ERRO`; `)` e 👍 reconhecidos
29	texto sem aspa final	`L3`, coluna 15	a linha 30 é analisada normalmente, com o seu `STRING`
33	comentário de bloco aberto	`L7`, linha 33, coluna 1	apontado na abertura; a análise vai até o `EOF`

Os 19 erros esperados foram relatados, com a linha, a coluna e o lexema corretos, e nenhum erro a mais apareceu: nenhum token válido foi perdido ou criado por causa de um erro vizinho. No modo compilador, o mesmo arquivo interrompe a compilação no primeiro erro:

```
$ bin/padolang check testes/lexer_editor/erros_lexicos.padoin
padoERRO léxico (linha 5, coluna 26): caractere inválido '@' (U+0040)
```

5.3 Testes automatizados

O roteiro `testar-lexer-editor.sh` executa o analisador em cada `.padoin` de `testes/lexer_editor/` e compara a saída, caractere a caractere, com o `.tokens` correspondente. Ele é executado junto com as outras suítes do repositório a cada alteração enviada.

```
$ ./testar-lexer-editor.sh
[ OK ]  testes/lexer_editor/cedilha_fora_de_texto.padoin
[ OK ]  testes/lexer_editor/comentario_de_bloco.padoin
[ OK ]  testes/lexer_editor/comentario_nao_fechado.padoin
[ OK ]  testes/lexer_editor/comentarios.padoin
[ OK ]  testes/lexer_editor/erros_lexicos.padoin
[ OK ]  testes/lexer_editor/texto_nao_fechado.padoin
[ OK ]  testes/lexer_editor/todos_os_erros.padoin
[ OK ]  testes/lexer_editor/todos_os_tokens.padoin

8 de 8 casos passaram.
```

As demais suítes do repositório continuam passando depois das alterações feitas para esta atividade: os 60 programas de `testes/` e `exemplos/` (compilados e executados), a linha de comando, o formatador e o servidor de linguagem.

5.4 Conferência da especificação com o JFlex

As expressões regulares da Seção 2 estão no arquivo `padolang.flex`, com as palavras reservadas antes do identificador e os padrões de lexemas malformados depois dos válidos. O roteiro `conferir.sh` gera um scanner com o JFlex 1.9.1 a partir desse arquivo e compara a listagem de tokens dele com a do `Lexer.java`: classe, lexema, linha e coluna de cada token, inclusive dos trechos inválidos. A comparação é feita em todos os programas `.padoin` do repositório e em 300 000 entradas aleatórias, montadas com pedaços que provocam os conflitos e os erros do léxico (prefixos de palavras reservadas, dígitos e pontos, aspas e barras, `#` e `*`, operadores, quebras de linha, 👍 e símbolos inválidos).

```
$ JFLEX_CP=jflex-1.9.1.jar:java-cup-runtime-11b-20160615-3.jar docs/estudo-independente-03/jflex/conferir.sh
75 de 75 arquivos .padoin com a mesma listagem de tokens
0 diferenças em 300000 entradas aleatórias
```

Não houve nenhuma diferença. Isso mostra que a especificação formal do Problema 1 e a implementação manual do Problema 3 definem os mesmos tokens, e que as regras de prioridade e de maior casamento codificadas à mão (Problema 2) dão o mesmo resultado que as do JFlex.

6 Considerações finais

O analisador léxico da Padolang reconhece 48 classes de token, todas descritas por expressões regulares. A implementação é um autômato finito determinístico escrito à mão, que precisa de um único símbolo de *lookahead*. A prioridade das palavras reservadas é dada pela consulta a uma tabela depois de lido o identificador inteiro; o maior casamento, pelo teste do próximo símbolo antes de emitir o token; e a posição, por um único método que consome símbolos e atualiza linha e coluna.

Escrever o analisador à mão deu controle sobre dois pontos que um gerador não resolve sozinho: a contagem de colunas em *code points*, necessária por causa do 👍, e as mensagens de erro, com um código, um trecho e uma estratégia de recuperação para cada tipo de lexema malformado. O custo é que a correspondência entre a especificação e o código deixa de ser automática; por isso a especificação foi escrita em JFlex e executada contra o analisador.

Limitações conhecidas: o comentário de bloco não aninha; não há notação científica para números reais; e, pelo maior casamento, `5--3` é lido como decremento, o que exige escrever `5 - -3`.

Referências

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compiladores**: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008. Capítulo 3: Análise léxica.

KLEIN, Gerwin; ROWE, Steve; DÉCAMPS, Régis. **JFlex user's manual**: version 1.9.1. 2023. Disponível em: https://jflex.de/manual.html. Acesso em: 8 out. 2026.

MENEZES, Paulo Blauth. **Linguagens formais e autômatos**. 6. ed. Porto Alegre: Bookman, 2011. Capítulo 3: Linguagens regulares.

PRICE, Ana Maria de Alencar; TOSCANI, Simão Sirineo. **Implementação de linguagens de programação**: compiladores. 2. ed. Porto Alegre: Instituto de Informática da UFRGS: Sagra Luzzatto, 2001. Capítulo 2: Análise léxica.

Declaração de uso de IA generativa

Declaramos que utilizamos inteligência artificial generativa na elaboração deste trabalho, da forma descrita a seguir.

Ferramenta. Claude Code (Anthropic), com o modelo Claude Opus 5.5, usado como assistente de programação e de escrita dentro do repositório do compilador da Padolang.

Em que a ferramenta foi usada.

1. **Código-fonte.** Para esta atividade, a ferramenta implementou o comentário de bloco (com o erro `L7` quando não é fechado), o erro único para números com mais de um ponto decimal, a exibição do lexema em cada erro da listagem de tokens e os ajustes que o comentário de bloco exigiu no formatador, no servidor de linguagem e nas gramáticas dos editores.
2. **Casos de teste.** A ferramenta escreveu os arquivos de teste criados para esta atividade (`todos_os_tokens`, `erros_lexicos`, `comentario_de_bloco` e `comentario_nao_fechado`) e gerou as saídas esperadas executando o analisador.
3. **Relatório.** A ferramenta redigiu a primeira versão do texto, das tabelas e dos diagramas de transição, com base no código, nas decisões de projeto registradas no repositório e na bibliografia indicada no enunciado.

Como o resultado foi conferido. Todas as listagens de saída do relatório foram produzidas pela execução real do analisador. As suítes de teste do repositório foram executadas depois das alterações e passam. A especificação em JFlex foi executada e comparada com o analisador escrito à mão em todos os programas do repositório e em 300 000 entradas aleatórias, sem diferenças.

Responsabilidade. Os autores leram e revisaram o código, os testes e o texto produzidos com a ajuda da ferramenta, conferiram as citações na bibliografia e respondem pelo conteúdo entregue, inclusive por eventuais erros.

Theodoro Gaspar Ferreira
Vinícius Andrey Costa Mattos

Apêndice A — Código-fonte do analisador (`Lexer.java`)

Arquivo `src/padolang/lexer/Lexer.java`, completo. Os demais arquivos do pacote `lexer` e o restante do compilador estão no pacote entregue.

```
  1  package padolang.lexer;
  2
  3  import padolang.diagnostico.Diagnostico;
  4  import padolang.diagnostico.Fase;
  5
  6  import java.util.ArrayList;
  7  import java.util.List;
  8  import java.util.Map;
  9
 10  /**
 11   * Analisador léxico da Padolang, escrito à mão.
 12   *
 13   * O fonte é percorrido por code points Unicode (e não por char), porque o 👍
 14   * (U+1F44D) ocupa dois char em Java (par surrogate). Assim cada símbolo de Σ
 15   * corresponde a exatamente uma posição do vetor cp.
 16   *
 17   * Dois modos:
 18   *   - COMPILADOR (padrão): comentários são descartados e o primeiro erro lança
LexicoException;
 19   *   - EDITOR (formatter, LSP e "padolang tokens --editor"): comentários viram tokens
COMENTARIO e
 20   *     cada erro vira um token ERRO, com o diagnóstico guardado em erros(); a análise continua
depois
 21   *     do trecho inválido.
 22   * Nos dois modos os demais tokens são os mesmos.
 23   */
 24  public class Lexer {
 25
 26      public enum Modo { COMPILADOR, EDITOR }
 27
 28      private static final int JOINHA = 0x1F44D;           // 👍
 29      private static final int TOM_PELE_INICIO = 0x1F3FB;  // 🏻 ... 🏿
 30      private static final int TOM_PELE_FIM = 0x1F3FF;
 31      private static final int SELETOR_VARIACAO = 0xFE0F;
 32      private static final int BOM = 0xFEFF;               // marca de ordem de bytes (editores
do Windows)
 33
 34      /** Palavras reservadas: o identificador é lido inteiro (maior casamento) e só depois
consultado aqui. */
 35      private static final Map<String, TokenType> RESERVADAS = Map.ofEntries(
 36              Map.entry("PADOIN", TokenType.PROGRAMA),
 37              Map.entry("padoin", TokenType.DECLARA),
 38              Map.entry("padoINTER", TokenType.TIPO_INTER),
 39              Map.entry("padoFLOAT", TokenType.TIPO_FLOAT),
 40              Map.entry("padoOLEAN", TokenType.TIPO_OLEAN),
 41              Map.entry("padoSTRING", TokenType.TIPO_STRING),
 42              Map.entry("padoIN", TokenType.LEITURA),
 43              Map.entry("padoPRINT", TokenType.ESCRITA),
 44              Map.entry("padoPRINTLN", TokenType.ESCRITA_LINHA),
 45              Map.entry("pad", TokenType.SE),
 46              Map.entry("oin", TokenType.SENAO),
 47              Map.entry("papapadoin", TokenType.REPETICAO),
 48              Map.entry("padoinquanto", TokenType.ENQUANTO),
 49              Map.entry("padBREAK", TokenType.INTERROMPE),
 50              Map.entry("padCONTINUE", TokenType.PROSSEGUE),
 51              Map.entry("padUNCTION", TokenType.FUNCAO),
 52              Map.entry("padOUT", TokenType.RETORNO),
 53              Map.entry("padoTRUE", TokenType.VERDADEIRO),
 54              Map.entry("padoFALSE", TokenType.FALSO),
 55              Map.entry("padAND", TokenType.E),
 56              Map.entry("padOR", TokenType.OU),
 57              Map.entry("padNOT", TokenType.NAO),
 58              Map.entry("padoMOD", TokenType.MODULO)
```

```
59      );
60
61      private final int[] cp;
62      private final Modo modo;
63      private int pos = 0;
64      private int linha = 1;
65      private int coluna = 1;
66      private int inicio;   // índice em cp do primeiro code point do lexema atual
67      private final List<Token> tokens = new ArrayList<>();
68      private final List<Diagnostico> erros = new ArrayList<>();
69
70      public Lexer(String fonte) {
71          this(fonte, Modo.COMPILADOR);
72      }
73
74      public Lexer(String fonte, Modo modo) {
75          this.cp = fonte.codePoints().toArray();
76          this.modo = modo;
77          // Um BOM no início do arquivo não faz parte do programa: é ignorado sem contar coluna
78          if (cp.length > 0 && cp[0] == BOM) {
79              pos = 1;
80          }
81      }
82
83      public List<Token> tokenizar() {
84          while (!fim()) {
85              int c = atual();
86
87              if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
88                  avancar();
89                  continue;
90              }
91
92              int l = linha, col = coluna; // posição de início do lexema
93              inicio = pos;
94
95              if (c == '#') {
96                  comentario(l, col);
97              } else if (ehLetra(c)) {
98                  identificadorOuReservada(l, col);
99              } else if (ehDigito(c)) {
100                 numero(l, col);
101             } else if (c == '"') {
102                 string(l, col);
103             } else if (c == JOINHA) {
104                 joinha(l, col);
105             } else {
106                 operadorOuDelimitador(l, col);
107             }
108         }
109         tokens.add(new Token(TokenType.EOF, "<fim>", linha, coluna, 0));
110         return tokens;
111     }
112
113     // ------------------------------------------------------------------ padrões
114
115     /** Diagnósticos léxicos encontrados no modo EDITOR (no modo COMPILADOR fica sempre vazia).
*/
116     public List<Diagnostico> erros() {
117         return erros;
118     }
119
120     /**
121      * Comentários. Os dois padrões começam por '#', e o símbolo seguinte decide qual é (um
símbolo
122      * de lookahead):
123      *   linha:  #([^*\n][^\n]*)?           — até o fim da linha;
124      *   bloco:  #\*([^*]|\*+[^*#])*\*+#     — de "#*" até o primeiro "*#", em uma ou mais
linhas.
125      * São descartados, ou viram um token COMENTARIO no modo EDITOR.
126      */
```

```
127      private void comentario(int l, int col) {
128          avancar(); // '#'
129          if (!fim() && atual() == '*') {
130              comentarioDeBloco(l, col);
131              return;
132          }
133          while (!fim() && atual() != '\n') {
134              avancar();
135          }
136          if (modo == Modo.EDITOR) {
137              emitir(TokenType.COMENTARIO, trecho(), l, col);
138          }
139      }
140
141      /** O resto do comentário de bloco, depois do '#'. Não aninha: fecha no primeiro "*#". */
142      private void comentarioDeBloco(int l, int col) {
143          avancar(); // '*'
144          while (!fim()) {
145              if (avancar() == '*' && casar('#')) {
146                  if (modo == Modo.EDITOR) {
147                      emitir(TokenType.COMENTARIO, trecho(), l, col);
148                  }
149                  return;
150              }
151          }
152          // O erro aponta o "#*" que ficou aberto; no modo EDITOR o trecho até o fim do arquivo
vira ERRO
153          falha("L7", "comentário de bloco não fechado (falta o '*#')", l, col, 2);
154      }
155
156      /** [a-zA-Z_][a-zA-Z0-9_]*  — depois consulta a tabela de reservadas. */
157      private void identificadorOuReservada(int l, int col) {
158          StringBuilder sb = new StringBuilder();
159          while (!fim() && (ehLetra(atual()) || ehDigito(atual()))) {
160              sb.appendCodePoint(avancar());
161          }
162          String lexema = sb.toString();
163          emitir(RESERVADAS.getOrDefault(lexema, TokenType.ID), lexema, l, col);
164      }
165
166      /** [0-9]+ (NUM_INT)  ou  [0-9]+\.[0-9]+ (NUM_FLOAT): no máximo um ponto, com dígitos dos
dois lados. */
167      private void numero(int l, int col) {
168          StringBuilder sb = new StringBuilder();
169          lerDigitos(sb);
170
171          TokenType tipo = TokenType.NUM_INT;
172          if (!fim() && atual() == '.') {
173              sb.appendCodePoint(avancar());
174              if (fim() || !ehDigito(atual())) {
175                  falha("L1", "número real malformado '" + sb + "' (esperado dígito após o
ponto)", l, col, pos - inicio);
176                  return;
177              }
178              lerDigitos(sb);
179              tipo = TokenType.NUM_FLOAT;
180
181              if (!fim() && atual() == '.') {
182                  // 1.2.3: o lexema inteiro é um erro só, em vez de um real seguido de um '.'
solto
183                  while (!fim() && (ehDigito(atual()) || atual() == '.')) {
184                      avancar();
185                  }
186                  falha("L1", "número real malformado '" + trecho() + "' (mais de um ponto
decimal)", l, col, pos - inicio);
187                  return;
188              }
189          }
190
191          if (!fim() && ehLetra(atual())) {
192              // O trecho apontado vai até o fim do "identificador" (ex.: 2x inteiro)
```

```
193             int fimTrecho = pos;
194             while (fimTrecho < cp.length && (ehLetra(cp[fimTrecho]) ||
ehDigito(cp[fimTrecho]))) {
195                 fimTrecho++;
196             }
197             falhaSemToken("L2", "lexema inválido '" + new String(cp, inicio, fimTrecho -
inicio)
198                     + "' (identificadores não podem começar com dígito)", l, col, fimTrecho -
inicio);
199             while (pos < fimTrecho) {
200                 avancar();
201             }
202             emitir(TokenType.ERRO, trecho(), l, col);
203             return;
204         }
205         emitir(tipo, sb.toString(), l, col);
206     }
207
208     /** "( [^"\\\n] | \\[nt"\\] )*"  — o lexema guarda aspas e escapes, compatíveis com C. */
209     private void string(int l, int col) {
210         StringBuilder sb = new StringBuilder();
211         sb.appendCodePoint(avancar()); // aspa de abertura
212         boolean escapeInvalido = false;   // só chega a true no modo EDITOR
213
214         while (true) {
215             if (fim() || atual() == '\n') {
216                 // No modo EDITOR o trecho até o fim da linha vira ERRO e a análise segue na
próxima linha
217                 falha("L3", "string não fechada", l, col, semCr(pos) - inicio);
218                 return;
219             }
220             int c = avancar();
221             if (c == '"') {
222                 sb.append('"');
223                 break;
224             }
225             if (c == '\\') {
226                 if (fim() || "nt\"\\".indexOf(atual()) < 0) {
227                     boolean temProximo = !fim() && atual() != '\n';
228                     falhaSemToken("L4", "sequência de escape inválida na string", linha, coluna
- 1,
229                             temProximo ? 2 : 1);
230                     // Modo EDITOR: pula o escape e continua; a string inteira vira ERRO no fim
231                     escapeInvalido = true;
232                     if (temProximo) {
233                         avancar();
234                     }
235                     continue;
236                 }
237                 sb.append('\\').appendCodePoint(avancar());
238             } else {
239                 sb.appendCodePoint(c);
240             }
241         }
242         if (escapeInvalido) {
243             emitir(TokenType.ERRO, trecho(), l, col);
244         } else {
245             emitir(TokenType.STRING, sb.toString(), l, col);
246         }
247     }
248
249     /** 👍 com modificador de tom de pele e seletor de variação opcionais. */
250     private void joinha(int l, int col) {
251         avancar();
252         if (!fim() && atual() >= TOM_PELE_INICIO && atual() <= TOM_PELE_FIM) {
253             avancar();
254         }
255         if (!fim() && atual() == SELETOR_VARIACAO) {
256             avancar();
257         }
258         emitir(TokenType.JOINHA, "👍", l, col);
```

```
259      }
260
261      /** Operadores com prefixo comum resolvidos por maior casamento: ++ antes de +, == antes de
=, etc. */
262      private void operadorOuDelimitador(int l, int col) {
263          int c = avancar();
264          switch (c) {
265              case '+' -> emitir(casar('+') ? TokenType.INCREMENTO : TokenType.MAIS, l, col);
266              case '-' -> emitir(casar('-') ? TokenType.DECREMENTO : TokenType.MENOS, l, col);
267              case '*' -> emitir(TokenType.VEZES, l, col);
268              case '/' -> emitir(TokenType.DIVIDIDO, l, col);
269              case '=' -> emitir(casar('=') ? TokenType.IGUAL : TokenType.ATRIB, l, col);
270              case '<' -> emitir(casar('=') ? TokenType.MENOR_IGUAL : TokenType.MENOR, l, col);
271              case '>' -> emitir(casar('=') ? TokenType.MAIOR_IGUAL : TokenType.MAIOR, l, col);
272              case '!' -> {
273                  if (casar('=')) {
274                      emitir(TokenType.DIFERENTE, l, col);
275                  } else {
276                      falha("L5", "'!' isolado não é um operador válido (use !=)", l, col, 1);
277                  }
278              }
279              case '(' -> emitir(TokenType.ABRE_PAR, l, col);
280              case ')' -> emitir(TokenType.FECHA_PAR, l, col);
281              case '{' -> emitir(TokenType.ABRE_CHAVE, l, col);
282              case '}' -> emitir(TokenType.FECHA_CHAVE, l, col);
283              case ',' -> emitir(TokenType.VIRGULA, l, col);
284              default -> falha("L6",
285                      "caractere inválido '" + Character.toString(c) + "' (U+" +
String.format("%04X", c) + ")", l, col, 1);
286          }
287      }
288
289      // ------------------------------------------------------------------ erros
290
291      /**
292       * Erro léxico. No modo COMPILADOR lança a exceção; no modo EDITOR registra o diagnóstico e
emite
293       * um token ERRO com o trecho já consumido desde o início do lexema.
294       */
295      private void falha(String codigo, String mensagem, int l, int col, int tamanho) {
296          falhaSemToken(codigo, mensagem, l, col, tamanho);
297          emitir(TokenType.ERRO, trecho(), l, col);
298      }
299
300      /** Como falha(), mas sem emitir o token (usado quando o lexema ainda não terminou). */
301      private void falhaSemToken(String codigo, String mensagem, int l, int col, int tamanho) {
302          Diagnostico d = Diagnostico.erro(Fase.LEXICO, codigo, l, col, tamanho, mensagem);
303          if (modo == Modo.COMPILADOR) {
304              throw new LexicoException(d);
305          }
306          erros.add(d);
307      }
308
309      // ------------------------------------------------------------------ auxiliares
310
311      private boolean fim() { return pos >= cp.length; }
312
313      private int atual() { return cp[pos]; }
314
315      private int avancar() {
316          int c = cp[pos++];
317          if (c == '\n') {
318              linha++;
319              coluna = 1;
320          } else {
321              coluna++;
322          }
323          return c;
324      }
325
326      /** Consome o próximo símbolo se for o esperado (usado no maior casamento). */
```

```
327      private boolean casar(int esperado) {
328          if (!fim() && atual() == esperado) {
329              avancar();
330              return true;
331          }
332          return false;
333      }
334
335      private void lerDigitos(StringBuilder sb) {
336          while (!fim() && ehDigito(atual())) {
337              sb.appendCodePoint(avancar());
338          }
339      }
340
341      /** Σ das letras: apenas ASCII. 'ç', 'ã' etc. fora de strings são erro léxico. */
342      private static boolean ehLetra(int c) {
343          return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_';
344      }
345
346      private static boolean ehDigito(int c) {
347          return c >= '0' && c <= '9';
348      }
349
350      private void emitir(TokenType tipo, String lexema, int l, int col) {
351          tokens.add(new Token(tipo, lexema, l, col, tamanho(lexema, tipo)));
352      }
353
354      /** O tamanho é o do trecho consumido; em COMENTARIO e ERRO o '\r' final (CRLF) fica de
fora. */
355      private int tamanho(String lexema, TokenType tipo) {
356          return (tipo == TokenType.COMENTARIO || tipo == TokenType.ERRO) ? semCr(pos) - inicio :
pos - inicio;
357      }
358
359      /** O texto do fonte desde o início do lexema atual, sem o '\r' de um fim de linha CRLF. */
360      private String trecho() {
361          return new String(cp, inicio, semCr(pos) - inicio);
362      }
363
364      /** Recua um índice de fim de trecho para antes de um '\r' final. */
365      private int semCr(int fimTrecho) {
366          return (fimTrecho > inicio && cp[fimTrecho - 1] == '\r') ? fimTrecho - 1 : fimTrecho;
367      }
368
369      /** Para operadores/delimitadores o lexema é recuperado do próprio fonte. */
370      private void emitir(TokenType tipo, int l, int col) {
371          StringBuilder sb = new StringBuilder();
372          int tamanho = (tipo == TokenType.INCREMENTO || tipo == TokenType.DECREMENTO
373                  || tipo == TokenType.IGUAL || tipo == TokenType.DIFERENTE
374                  || tipo == TokenType.MENOR_IGUAL || tipo == TokenType.MAIOR_IGUAL) ? 2 : 1;
375          for (int i = pos - tamanho; i < pos; i++) {
376              sb.appendCodePoint(cp[i]);
377          }
378          emitir(tipo, sb.toString(), l, col);
379      }
380  }
```